HARSHAVARDHAN NIMMAKANTI

✉ harshavardhan5530@gmail.com
✆ **913-703-6017**

PROFESSIONAL SUMMARY

- Senior Java Full Stack Developer with over 10 years of experience building scalable enterprise applications across Banking, Healthcare, and Insurance domains.
- Expertise in Core Java (8–21), J2EE, and frameworks including Spring Boot, Spring MVC, Spring Cloud, and Spring Data JPA.
- Designed and implemented microservices architecture leveraging RESTful APIs, GraphQL, and gRPC for distributed and high-performance systems.
- Strong experience with Hibernate, JPA, and advanced ORM techniques for transaction management and data integrity.
- Skilled in React.js, Angular (2–17), TypeScript, JavaScript (ES6+), HTML5, CSS3, Bootstrap, and Tailwind CSS for building responsive front-end interfaces.
- Applied Redux, NgRx, and RxJS for efficient state management and real-time data synchronization.
- Experienced with Kafka, RabbitMQ, IBM MQ, and Pulsar for asynchronous communication, messaging, and event-driven systems.
- Proficient in writing SQL and PL/SQL queries, stored procedures, and indexing across Oracle, PostgreSQL, MySQL, and SQL Server databases.
- Hands-on experience with NoSQL databases like MongoDB, Cassandra, Redis, and DynamoDB for distributed data storage and caching.
- Built and deployed applications on AWS (EC2, ECS, EKS, Lambda, RDS, API Gateway, S3, IAM), Azure (AKS, Blob Storage, Functions, Cosmos DB), and GCP (Compute Engine, Pub/Sub, Cloud SQL).
- Implemented Infrastructure as Code using Terraform, CloudFormation, Pulumi, and Ansible for automated provisioning and configuration management.
- Containerized and orchestrated services with Docker, Kubernetes, and Helm to achieve reliable and scalable deployments.
- Developed CI/CD pipelines using Jenkins, GitLab CI/CD, GitHub Actions, Azure DevOps, and AWS CodePipeline for automated build, test, and deployment processes.
- Practiced Test-Driven Development (TDD) and Behavior-Driven Development (BDD) using JUnit, Mockito, Cucumber, Selenium, Cypress, and Playwright.
- Integrated observability and monitoring with Datadog APM, Prometheus, Grafana, ELK Stack, and OpenTelemetry for performance and root cause analysis.
- Applied Site Reliability Engineering (SRE) principles to improve availability, resilience, and performance of production systems.
- Secured enterprise applications using Spring Security, OAuth2, JWT, Okta, HTTPS, TLS, and SSO for authentication and authorization.
- Built event-driven data pipelines integrating Kafka with AWS and cloud storage systems for high-throughput streaming and reliability.
- Automated backend and deployment workflows using Python and Node.js scripting.
- Experienced in version control and collaboration using Git, GitHub, GitLab, Bitbucket, Jira, and Confluence.
- Implemented DevOps and Agile (Scrum, SAFe 6.0) methodologies for iterative development and continuous delivery.
- Utilized Resilience4J, Spring Boot Actuator, SonarQube, and static code analysis tools for maintainability and fault tolerance.
- Managed cloud deployments with auto-scaling, DNS routing (Route 53), and hybrid cloud infrastructure optimization.
- Provided mentorship, conducted code reviews, and contributed to end-to-end delivery of enterprise-grade full-stack solutions.

TECHNICAL SKILLS

- **Programming Languages:** C, C++, Java 17, Python, SQL, PL/SQL, TypeScript, JavaScript.
- **Web Technologies:** jQuery, Ajax, Angular JS, Angular 2+, React JS, Node JS, Express JS XML, HTML5, CSS3, JSON, JavaScript, Typescript Webpack, Babel.
- **J2EE Technologies:** Servlets, JSP, JPA, JSF, JDBC, Spring Boot, Struts, EJB, Java.
- **Frameworks:** Spring, Hibernate, Bootstrap, Tailwind CSS, Angular, React, Spring MVC, Spring Data JPA, Spring Cloud (Service Discovery, Eureka).
- **Database Systems:** Oracle 10g/21c, SQL Server, MySQL, PostgreSQL, Mongo DB, Cassandra, Redis.
- **Messaging Queue:** Apache Kafka, Rabbit MQ, IBM MQ.
- **Developer Tools:** (IDE) Eclipse, Atom, IntelliJ, Dreamweaver, Microsoft Visual Studio, PyCharm.
- **Cloud Platforms:** AWS, Azure, GCP, Open shift, RedHat.
- **Operating Systems:** Windows, Linux, Mac OS.

- **Build & Release Tools:** Maven, Gradle, Ant, Docker, Kubernetes, Jenkins.
- **Application Server:** IBM Web Sphere, Web Logic, Apache Tomcat.
- **Web Services:** JAX-RPC, SOAP, REST, Swagger, RESTful, Graph QL.
- **Security Technologies:** OAuth2.0, JWT, Okta, HTTPS, TLS, Spring Security.
- **Version Controls:** Git, Bit Bucket, GitHub.
- **Testing tools:** Selenium, Jest, JUnit, Mockito, Karma, Jasmine, Cucumber, Protractor, Mocha, Chai, Postman, JMeter, Cypress.
- **Methodologies:** TDD, BDD, Agile (SAFe 6.0), Scrum.
- **Monitoring and Other tools:** Prometheus, Grafana, ELK, Kibana, Jira, Elastic Search, Cloud watch.

PROFESSIONAL EXPERIENCE

Dignity Health: July 2023 - Current (Location: San Francisco, CA)

Sr. Full Stack Java Developer
Responsibilities:

- Developed responsive front-end applications using TypeScript and React 16 for healthcare management systems. The interface works seamlessly across various devices, significantly increasing user engagement among medical staff.
- Supported and enhanced existing enterprise-grade healthcare applications by designing, developing, and maintaining robust, scalable Java-based backend services and APIs using Java 17–21 and Spring Boot 3.x.
- Collaborated with UI/UX designers and frontend teams to integrate Angular and TypeScript components with backend logic, ensuring seamless UI communication and consistent user experience.
- Set up WebPack and Babel for build optimization and browser compatibility in medical software applications. Utilized NPM to manage dependencies and automate builds, streamlining the development process.
- Implemented Tailwind CSS to style React applications for patient management interfaces. Created a unified design system that accelerated development and maintained UI consistency across multiple healthcare platforms.
- Built reusable components with React hooks for electronic health record systems. This approach reduced development time and improved code maintainability in complex medical applications.
- Improved React for patient data management through memoization, lazy loading, Virtual DOM, and code splitting. These optimizations resulted in faster load times, crucial for quick access to medical information.
- Integrated Redux for state management in large-scale healthcare React applications. This implementation maintained data consistency and boosted performance in complex patient record systems.
- Implemented Redux middleware including redux-promise, redux-thunk, and redux-saga in medical data processing applications. These enhancements improved state management and overall application efficiency in handling large volumes of patient data.
- Utilized RxJS to manage reactive programming in healthcare monitoring systems. This implementation handled asynchronous data streams effectively, making complex React applications more responsive for real-time patient data updates.
- Designed and developed robust and scalable backend microservices using Java 21 and Spring Boot 3.x, ensuring optimized performance and modular architecture across healthcare applications.
- Developed microservices with Java 17 for modular healthcare systems. Implemented sealed classes, records, and virtual threads to create more efficient and maintainable code for various medical services.
- Optimized application performance through asynchronous request handling, caching strategies, and thread management using Virtual Threads for faster processing of patient data.
- Participated in the complete software development lifecycle , from requirements gathering to design, coding, testing, and deployment, ensuring end-to-end delivery of enterprise-grade healthcare systems.
- Created RESTful APIs with Spring MVC and developed microservices using Spring Boot for healthcare data exchange. Applied Spring Core annotations for Dependency Injection, improving service performance in handling medical records.
- Built and maintained modular microservices architecture using Spring Cloud and containerized deployments, ensuring high performance and scalability across distributed systems.
- Enhanced backend components to support new features and optimized legacy modules for better maintainability and throughput.
- Built and integrated RESTful APIs and Microservices architecture with Spring Boot 3.x, ensuring high availability, fault tolerance, and scalability.
- Configured Spring Boot Actuator and Resilience4J for real-time service health monitoring and circuit breaker patterns.
- Built application features using Spring Boot with Hibernate ORM for patient database management. Incorporated Spring IOC, transactions, and security modules to enhance performance and security measures for sensitive medical data.
- Implemented OAuth 2.0, HTTPS, and TLS security using Spring Security for centralized user authentication in healthcare systems. Managed JWT access tokens across multiple microservices to control authorization and ensure secure transfer of patient data.
- Integrated Single Sign-On (SSO) authentication using OAuth 2.0 and Spring Security, enabling unified access control across multiple healthcare microservices

- Utilized Express.js on Node.js to manage communication between RESTful services and Oracle Database for medical record systems. Added middleware and routing features to streamline data processing and enhance API performance for quick retrieval of patient information.
- Created and implemented middleware patterns in Node.js applications for healthcare data processing. Added error handling and request processing middleware to strengthen application reliability in critical medical systems.
- Generated API documentation with Swagger for healthcare service endpoints. Used code annotations to create interactive documentation, helping developers understand and test API endpoints for various medical data exchange services.
- Developed Oracle database components including triggers, functions, and packages with PL/SQL for medical data management. Wrote code to handle business logic and process patient data automatically within the database.
- Authored PL/SQL code for Oracle databases, implementing CRUD operations, database Views, table Joins, Indexes, and Stored Procedures. These optimizations made database queries faster and more efficient for accessing and updating patient records.
- Implemented Kafka's publish-subscribe system to decouple message producers from consumers in healthcare data streams. Added message queuing and event streaming to enable microservices to work independently and handle increased traffic in patient data processing.
- Collaborated closely with DevOps and QA teams to design CI/CD pipelines, automate builds and testing using Jenkins, GitHub Actions, and Docker, and integrate automated regression testing in production pipelines.
- Debugged and resolved production issues by leveraging Datadog APM, ELK Stack, and OpenTelemetry for distributed tracing and root cause analysis.
- Constructed real-time data streaming systems with Kafka to process and transfer large volumes of medical data into Oracle databases. Set up producers and consumers to efficiently handle continuous flow of patient information and medical device data.
- Integrated and managed RabbitMQ messaging queues for asynchronous communication between microservices, improving reliability and performance in data synchronization.
- Tuned Kafka consumer groups and partition strategies to support large-volume real-time medical data streams.
- Designed event-driven systems using Kafka to connect and synchronize data between Oracle databases and AWS cloud services for healthcare applications. Implemented message brokers and event handlers for reliable transfer of medical records and diagnostic data.
- Designed event-driven systems using Kafka to connect and synchronize data between Oracle databases and AWS cloud services for healthcare applications. Implemented message brokers and event handlers for reliable transfer of medical records and diagnostic data.
- Automated Kafka cluster provisioning and AWS infrastructure using Terraform, ensuring consistent configuration and streamlined environment setup for high-availability data streaming systems.
- Engineered and operated Confluent Kafka clusters on AWS EKS, supporting high-throughput data streams for real-time healthcare analytics.
- Implemented Observability and Distributed Tracing using Datadog APM, OpenTelemetry, and ELK stack, enhancing root-cause analysis and event logging capabilities.
- Applied SRE practices to improve system reliability, uptime metrics, and incident response automation.
- Utilized Ansible for infrastructure configuration management and deployment automation across hybrid cloud environments.
- Developed Python scripts for data processing and system management tasks in healthcare environments. Added automation for file handling and backend operations to expedite routine tasks in medical data management.
- Created Python scripts for batch processing to handle large sets of medical data efficiently. Incorporated error handling and logging to ensure reliable processing of patient information with minimal system interruptions.
- Configured AWS IAM roles and policies for secure access control in healthcare cloud infrastructure. Utilized Route 53 DNS services to manage domain names and control traffic routing across various medical applications.
- Employed Amazon Elastic Beanstalk to deploy and run web applications for patient portals and healthcare management systems. Set up AWS Lambda functions and API Gateway to create serverless microservices that handle both RESTful and SOAP web service requests for medical data processing.
- Utilized GitHub for code version control in collaborative healthcare software development. Implemented branching strategies and review processes to maintain code stability and reduce conflicts in critical medical applications.
- Established automated workflows with GitHub Actions to manage continuous integration for healthcare software. Created pipelines for automated testing, building, and deploying code to improve development speed and reliability of medical systems.
- Developed and deployed unified CI/CD workflows connecting GitHub Actions and Jenkins pipelines for containerized healthcare applications. Set up automated triggers for builds and testing from GitHub pull requests, significantly improving delivery speed of medical software updates.
- Constructed comprehensive CI/CD pipelines using Jenkins integrated with Docker for healthcare application deployment. Implemented systematic testing at each stage, substantially reducing environment inconsistencies and deployment failures in critical medical systems.
- Built CI/CD pipelines for continuous integration and delivery using Jenkins and GitHub Actions, incorporating automated testing and code-quality checks to ensure faster and reliable releases.
- Deployed containerized applications on AWS EKS/ECS using Docker and Kubernetes orchestration, improving scalability and reducing manual deployment efforts.
- Configured secure Jenkins pipeline setups to create Docker images for healthcare applications and automatically store them in Amazon ECR. Developed monitoring systems to track image versions and implemented cleanup procedures for outdated container images in medical software deployments.

- Established and managed container deployments on AWS ECS/EC2 infrastructure for healthcare services, implementing auto-scaling policies and high-availability configurations. Created monitoring dashboards and alerts, leading to faster deployment cycles and improved reliability of medical data systems.
- Continuously improved application reliability, performance, and scalability through code reviews, deployment automation, and proactive monitoring—maintaining alignment with enterprise DevOps, CI/CD, and SRE best practices.
- Built and enhanced Angular 15+ interfaces using TypeScript, HTML5, and CSS3 with Angular CLI for dynamic healthcare dashboards, improving front-end performance and responsiveness across patient-management modules.
- Integrated Spring MVC controllers with RESTful APIs to manage asynchronous communication between front-end Angular applications and backend microservices, ensuring secure and efficient data flow.
- Designed SOAP-based web service endpoints alongside REST APIs to integrate with legacy healthcare systems and ensure interoperability between cloud and on-premise components.
- Implemented JUnit and Jasmine test cases for both Java and Angular modules, achieving high test coverage and ensuring consistent code reliability through automated CI/CD validation.
- Optimized backend microservices for scalability and throughput using Java 21 virtual threads, asynchronous calls, and Spring Boot 3.x configuration tuning for large healthcare datasets.
- Collaborated in Agile sprint cycles using Jira and Confluence for task tracking, backlog grooming, and sprint planning, maintaining timely delivery of healthcare features.
- Mentored junior developers on Angular component design, Spring MVC patterns, and debugging best practices to enhance project productivity and maintain quality coding standards.

Environment: Bootstrap, HTML5, CSS3, JavaScript, TypeScript, DOM, XHTML, AJAX, jQuery, React, Redux, Angular, Spring MVC, Hibernate, Spring Boot, TDD, Gradle, Microservices, RESTful APIs, SOAP, RabbitMQ, Kafka, Docker, Kubernetes, Terraform, Jenkins, AWS (CodePipeline, CloudFormation, Lambda, SQS, IAM, EKS, ECS), Oracle, Cassandra, PL/SQL, CI/CD, Jira, Git, JMS, Resilience4J, Spring Actuator, OAuth 2.0, JWT, Python, Ansible, Datadog APM, OpenTelemetry, ELK Stack, Terraform, AWS Beanstalk, Route 53, SSO, Spring Security.

Bank of America: September 2020 - June 2023 (Location: Charlotte, NC)

Sr Full Stack Java Developer
Responsibilities:

- Worked in an Agile team using SCRUM methodology for financial software development, participating in all phases including requirements gathering, analysis, design, development, and testing. Implemented test-driven development (TDD) practices to ensure code quality and reliability in banking applications.
- Designed, developed, and maintained end-to-end web applications using Java 11, Spring Boot, and Node.js with TypeScript to support secure and scalable financial services.
- Built and consumed RESTful APIs and microservices, integrating them with Angular 15–based user interfaces to deliver responsive, high-performance online banking applications.
- Implemented responsive UI designs using Angular and ensured seamless user experiences across desktop and mobile platforms.
- Worked with relational and NoSQL databases, including Oracle, MySQL, and MongoDB, for large-scale financial data management and reporting.Debugged, optimized, and automated application modules and CI/CD pipelines to improve performance, reliability, and developer efficiency.
- Collaborated with cross-functional teams including QA, DevOps, and Product to deliver features on schedule and maintain production quality standards.
- Built responsive user interfaces for online banking platforms using JSP, Bootstrap, JavaScript, TypeScript, XML, XHTML, XSL, and XSLT technologies. Developed robust business logic implementations using Servlets, JSP, and J2EE framework components for secure financial transactions.
- Created interactive single-page applications (SPAs) with Angular for wealth management portals that utilized two-way data binding, templates, and directives. These improvements significantly increased user engagement metrics and enhanced overall user experience for banking customers.
- Built and maintained modular, reusable UI components using Angular's component-based architecture for financial dashboards. This approach improved application scalability and substantially reduced the time spent on maintenance tasks for banking software.
- Implemented comprehensive form validation and user input handling using Angular reactive forms and observables for asynchronous operations in loan application systems. This rtlsesulted in achieving high successful form submission rates across the banking application.
- Modernized existing codebase by migrating legacy banking systems to Angular, utilizing Angular CLI, Angular Router, and Payment Processing for state management. These updates significantly improved application performance and maintainability metrics for financial services.
- Connected Angular frontend services with backend APIs using HttpClientModule for real-time financial data retrieval. This resulted in faster data retrieval times and improved application responsiveness for banking

customers.

- Implemented NgRx for state management across the banking application, incorporating Redux DevTools for enhanced debugging capabilities. This reduced the time needed to identify and fix bugs in complex financial transactions.
- Built responsive and consistent user interfaces using Angular Material for online banking platforms. This helped speed up the development process and improved overall user experience for customers accessing their accounts.
- Used Hibernate with Spring Framework to implement dependency injection and aspect-oriented programming in financial transaction processing systems. This created code that was easier to maintain and update for banking operations.
- Built applications using Java 11 and implemented the new HTTPClient API to replace older HttpURLConnection methods in banking communication systems. This improved code readability and made HTTP calls more reliable for secure financial data transfer.
- Created APIs with Spring MVC and Spring Cloud, working with Kafka to handle real-time data streaming needs for financial market updates. Managed message queuing and event processing to ensure reliable delivery of critical banking data.
- Built enterprise-level applications using Spring frameworks including Spring IOC, Spring MVC, Spring JDBC, Spring Annotations, Spring AOP, Spring Integration, and RESTful web services for comprehensive banking solutions. Ensured proper implementation of dependency injection and separation of concerns in financial software architecture.
- Created and refined Graph QL schemas to handle complex queries and mutations for investment portfolio management. Made improvements to query performance and response times through schema optimization for financial data retrieval.
- Connected Graph QL with existing RESTful services to provide unified data access across different banking platforms. Implemented efficient data fetching strategies to minimize response times for customer account information.
- Added security features to Graph QL APIs using OAuth2.0 and JWT for authentication and authorization in banking systems. This ensured secure access to sensitive financial data while meeting industry security standards for online banking.
- Worked with various teams to create data models, design database schemas, and build data access layers in MongoDB for customer relationship management. Ensured the database design matched business needs and maintained consistent data handling practices for banking operations.
- Improved MongoDB performance through proper indexing and sharding strategies for large-scale financial data storage. Optimized database queries to handle high-volume data requests efficiently in banking transactions.
- Created and implemented Kafka connectors to link MongoDB with Kafka topics for real-time financial data processing. Set up data pipelines for efficient ingestion, synchronization, and processing of data streams in banking systems.
- Implemented Kafka as a messaging system in Azure environments to handle high volumes of financial transactions with fault tolerance. Set up real-time data streaming pipelines to process banking information efficiently and securely.
- Created UNIX-based solutions to process customer transaction records in batches for end-of-day reconciliation. Improved the efficiency of data handling processes while reducing processing errors in financial reporting.
- Wrote UNIX scripts to combine card transaction data with CRM systems and monitor system resources for banking operations. Enhanced data consistency and improved the quality of customer service delivery in financial services.
- Managed and engineered Azure Virtual Machines to provide scalable computing resources for banking applications, successfully reducing system downtime and maintaining an industry-leading uptime service level agreement for online banking services.
- Created data storage solutions using Azure Blob Storage for secure retention of financial records, carefully optimizing for both scalability and cost-effectiveness. This resulted in a significant reduction in storage expenses and enabled faster data retrieval for customer account information.
- Deployed Azure Active Directory for comprehensive access management in banking systems, seamlessly integrating with Azure DevOps and Azure Kubernetes Service. This established secure and efficient user access across development environments for financial software development.
- Designed and implemented scalable RESTful APIs and Spring Boot 2.x microservices supporting secure financial data exchange, improving system modularity and throughput.
- Utilized Java 11 concurrency utilities and multithreading techniques to optimize backend performance and reduce request-handling latency for high-volume banking transactions.
- Implemented key modules using Spring MVC, Spring Security, and dependency injection for consistent, secure, and maintainable codebases across distributed services.
- Worked extensively with Oracle and MySQL databases for transactional systems and MongoDB for unstructured data storage, ensuring efficient data persistence and retrieval.
- Automated build, integration, and deployment pipelines using Jenkins, Git, Maven, and Docker, enabling faster and reliable delivery across environments.
- Deployed and orchestrated containerized services using Kubernetes (AKS), achieving high availability and simplified scaling in Azure environments.
- Integrated backend microservices with Azure cloud components, ensuring fault tolerance, scalability, and cost optimization for banking applications.
- Applied rigorous unit testing and integration testing using JUnit and Mockito, ensuring over 90% test coverage and improving overall code quality.

- Collaborated within Agile (Scrum) teams, contributing to sprint planning, code reviews, and CI/CD improvements for continuous delivery cycles.
- Followed best practices aligned with Oracle Certified Java Developer standards to maintain clean, high-quality, and performance-tuned Java applications.
- Developed enterprise back-end modules in Java 11 using Spring Boot and Spring MVC to deliver highly scalable APIs supporting secure financial transactions and account services.
- Designed Angular front-end components using TypeScript, HTML5, and CSS3 integrated with RESTful APIs to provide interactive and real-time financial dashboards for clients.
- Created SOAP web services in J2EE for legacy integration between payment gateways and account management modules, ensuring system interoperability within financial ecosystems.
- Managed relational data in IBM DB2 and Teradata databases, implementing stored procedures, triggers, and performance tuning for faster data retrieval and transactional consistency.
- Performed database optimization and indexing in DB2 to improve query response time by 30% and reduce backend latency in financial reporting systems.
- Utilized JUnit, Mockito, and Jasmine frameworks to validate both front-end and back-end logic, ensuring secure and bug-free deployment in production environments.
- Maintained source code versioning and CI/CD integration using Git, GitHub Actions, and Jenkins to automate builds, testing, and deployments in Agile financial sprints.
- Contributed to architecture discussions on service scalability and application fault-tolerance, aligning with enterprise standards for high-availability financial systems.
- Provided technical mentorship to new developers on Spring, Angular, and database design practices, improving team onboarding and reducing development turnaround time.

Environment: Java 11, J2EE, HTML5, CSS3, Hibernate, JavaScript, jQuery, Angular 15, TypeScript, Bootstrap, Spring IOC, Spring MVC, Graph QL, Microservices, Mongo DB, UNIX, Kafka, XML, Azure (VM's, AKS, ACR, Functions, Blob storage, Active Directory), Azure DevOps, CI/CD, GitHub, GitHub Actions, Maven, JUnit, Jenkins, Docker, Kubernetes, Git, Mockito, Jira.

Workday: November 2017 - August 2020 (Location: Pleasanton, CA)

Full Stack Java Developer
Responsibilities:

- Implemented agile development methodologies throughout software development cycles, incorporating Hibernate for the persistence layer. Developed complex database interactions using Hibernate mappings with Annotations, crafting efficient HQL queries, Criteria queries, and optimized database access patterns for human resource management systems.
- Utilized Gradle as a comprehensive build management tool, creating advanced build configurations with custom tasks and plugins. Successfully managed project dependencies and automated the build process to improve development efficiency and project workflow for enterprise resource planning software.
- Developed enterprise-scale Java web applications with a robust front-end technology stack, including JavaScript, TypeScript, Bootstrap, HTML5, DOM manipulation, XHTML, AJAX, CSS3, and jQuery. Created interactive and responsive user interfaces that met complex business requirements for financial management and human capital management platforms.
- Transformed legacy codebase by implementing React Hooks and Context API, achieving a significant 30% reduction in page load times. Enhanced application performance and user experience through strategic code refactoring and modern React practices for employee self-service portals.
- Implemented Redux for state management in React applications, resolving state-related challenges and reducing application crashes by 40%. Established a robust, predictable state management approach that improved overall application stability for workforce planning tools.
- Integrated frontend React components with backend services using RESTful APIs, enabling seamless communication and data exchange. Optimized data retrieval processes, resulting in a 25% reduction in data retrieval time and improved system responsiveness for payroll processing systems.
- Utilized Core Java Collections, advanced Exception Handling, Multi-Threading, and Serialization techniques. Incorporated Java 8 features including Lambda expressions, Stream API, and Bulk data operations on Collections to enhance application performance, enable pipeline processing, and implement efficient method references in talent management modules.
- Implemented a microservices architecture using Spring MVC, which enabled horizontal scalability and improved resource utilization. This strategic approach allowed the system to efficiently handle increased user requests during peak hours, ensuring robust performance under high load for cloud-based HR solutions.
- Utilized Spring Boot Annotations to simplify business logic for creating RESTful web services. Implemented Spring Boot Actuator to monitor and externalize configuration properties across different environments, enhancing system

observability and configuration management for enterprise financial systems.

- Analyzed and resolved software issues identified by QA and production monitoring teams using Datadog APM, ELK Stack, and OpenTelemetry, ensuring timely issue resolution and minimal downtime.
- Optimized performance of Java microservices through advanced caching strategies and database query optimization techniques. Achieved a significant 20% reduction in response times, improving overall system responsiveness and user experience for time tracking and absence management features.
- Developed comprehensive CRUD operations and implemented data transfer mechanisms using RESTful web services, facilitating seamless communication between the controller layer and data resources using JSON format. This approach streamlined data management for employee benefit administration systems.
- Designed and constructed a scalable, high-performance data pipeline using Apache Cassandra and Java. Significantly reduced data processing latency and improved real-time data accessibility for advanced analytics platforms used in workforce analytics.
- Employed Cassandra's native query language (CQL) to optimize and streamline complex database queries, substantially improving query performance and database interaction efficiency for large-scale employee data management.
- Constructed a high-performance distributed data infrastructure utilizing Apache Cassandra for a large-scale data analysis platform. Successfully reduced query latency by 20%, enabling faster and more efficient data processing and analysis for company-wide reporting and analytics.
- Implemented Java Message Service (JMS) with RabbitMQ to manage complex message publishing, subscription, and routing processes. Developed robust messaging infrastructure to support intricate business process workflows and ensure reliable communication between system components in procurement and expense management modules.
- Designed and developed RabbitMQ connectors using Spring Integration to integrate with backend services. Created efficient message processing mechanisms that effectively reduced system load during peak traffic periods, ensuring optimal performance and responsiveness for global payroll systems.
- Utilized RabbitMQ for comprehensive inter-service communication within a microservices architecture. Implemented robust real-time data streaming solutions that enabled seamless and reliable messaging infrastructure across distributed system components for talent acquisition and onboarding processes.
- Developed serverless applications using AWS Lambda, creating scalable backend processes that eliminated traditional server management complexities. Integrated these serverless functions seamlessly with API Gateway to generate responsive and efficient RESTful API endpoints for employee performance management systems.
- Optimized application performance and cost efficiency through meticulous fine-tuning of memory allocation and timeout settings. Implemented AWS CloudWatch for comprehensive monitoring and logging, ensuring system visibility and performance tracking for critical HR and financial processes.
- Constructed CI/CD pipelines using Jenkins and AWS Code Pipeline to automate serverless application deployments. Integrated with Git version control systems to facilitate rapid, consistent, and reliable software release processes for continuous improvement of HR management tools.
- Automated infrastructure provisioning and deployment using AWS CloudFormation and Terraform. Established repeatable and consistent environment configurations that streamlined development and operational workflows for global workforce management systems.
- Ensured adherence to technology standards and coding best practices while enhancing existing modules and designing new components.
  • Documented system architecture, API design, and feature specifications before production rollout.
- Enhanced API security by implementing robust IAM roles, comprehensive policies, API keys, and Lambda authorizers. Ensured controlled access and maintained strict compliance with critical security standards and best practices for protecting sensitive employee and financial data.
- Configured Amazon SQS to manage high-throughput message ingestion and integrated with RabbitMQ for complex message routing and processing scenarios. Developed a resilient communication framework that facilitated reliable interactions between microservices in the compensation management system.
- Implemented CI/CD pipelines integrating Jenkins, Docker, and Kubernetes to automate software delivery processes. Achieved a 30% improvement in development efficiency by streamlining build, test, and deployment workflows for the core HR platform.
- Created comprehensive unit test cases using the Mockito framework to validate code accuracy and functionality. Implemented robust testing strategies that ensured precise verification of individual code components, improving overall software reliability and performance across the enterprise resource planning suite.
- Led technical discussions, conducted peer code reviews, and mentored junior developers on Java, Spring Boot, and microservices best practices.
- Continuously learned and adopted new technologies such as Resilience4J, Spring Actuator, and updated cloud-native tools to improve application reliability and performance.

Environment: Bootstrap, HTML5, CSS3, Typescript, DOM, XHTML, AJAX, jQuery, React, Redux, Spring MVC, Hibernate, Spring Boot, TDD, Gradle, Microservices, RESTful, SOAP, RabbitMQ, Docker, Terraform, Java/J2EE, Kubernetes, Jenkins, AWS (Code Pipeline, CloudFormation, Lambda, SQS, IAM), Cassandra, CI/CD, Jira, Git, JMS.

Equifax: January 2016 - October 2017 (Location: Atlanta, GA)

Java Developer
Responsibilities:

- Created the user interface using JSP, JavaScript, CSS, TypeScript, and HTML, while developing business logic through Servlets, JSP, and J2EE framework technologies. This comprehensive approach resulted in a seamless and responsive front-end for credit reporting and risk assessment tools.
- Implemented Spring Boot starters and auto-configuration to simplify development processes, significantly reducing the time spent on initial project setup and configuration. This streamlined approach accelerated the development of new features for identity verification systems.
- Connected front-end Angular components with backend services through RESTful APIs and Spring MVC, which improved data retrieval efficiency by 25% and enabled smoother inter-component communication. The enhanced connectivity optimized the performance of credit score monitoring applications.
- Built microservices with Spring Boot, creating a modular system that allows independent scaling of different application components, thereby increasing the overall system's flexibility and performance capabilities. This architecture improved the responsiveness of fraud detection services during high-traffic periods.
- Constructed REST APIs using Spring MVC to establish reliable communication channels between microservices. These APIs facilitated efficient data exchange for complex credit analysis and reporting processes.
- Set up Spring Boot configurations for comprehensive error handling and exception management, ensuring the application could gracefully respond to unexpected operational scenarios. This robust error management system enhanced the reliability of consumer dispute resolution platforms.
- Created and refined SQL queries and stored procedures in Oracle Database, which successfully improved data retrieval performance by 30% and significantly reduced overall query execution time. The optimized database interactions accelerated credit report generation and analysis.
- Constructed and implemented IBM MQ pipelines for microservices architecture, establishing robust data communication channels and ensuring consistent, reliable message delivery across the system. This messaging infrastructure supported real-time updates for credit monitoring alerts.
- Developed and deployed Java-based backend services on Google Cloud Platform (GCP), implementing strategies to maintain high system availability and enable seamless scalability. The cloud-based approach enhanced the performance of identity protection services during peak usage periods.
- Implemented Google Cloud Pub/Sub for managing asynchronous messaging between microservices, which enhanced system reliability and overall application performance. This messaging system improved the responsiveness of credit score change notifications.
- Established and optimized Google Cloud Storage configurations to support efficient data storage and retrieval processes, ensuring comprehensive data durability and continuous accessibility. The enhanced storage solution facilitated faster access to historical credit data for trend analysis.
- Implemented and managed Google Cloud SQL for handling relational databases, creating secure and streamlined data management solutions for Java-based applications. This database solution improved the efficiency of storing and retrieving sensitive consumer credit information.
- Utilized Spring Security to implement robust authentication and authorization mechanisms for protecting sensitive financial data. The enhanced security measures ensured compliance with industry regulations and safeguarded consumer information in credit reporting systems.
- Developed custom Java annotations to simplify code and improve maintainability across the credit monitoring platform. This approach reduced boilerplate code and enhanced the overall readability of the codebase for fraud detection algorithms.
- Implemented caching strategies using Redis to optimize frequently accessed credit report data, resulting in a 40% reduction in database load. The improved caching mechanism significantly enhanced the responsiveness of credit score lookup services.
- Developed and optimized Teradata queries, joins, and stored procedures for large-volume financial datasets, ensuring efficient ETL operations and high-performance analytics.
- Designed data models in IBM DB2 and integrated with Spring Data JPA to handle secure financial data processing within Java-based microservices.
- Performed advanced performance tuning and query optimization in DB2 for real-time credit reporting applications, improving database response efficiency and reducing bottlenecks.
- Created RESTful and SOAP APIs using Spring MVC and JAX-RS to integrate Equifax credit reporting systems with external risk-assessment services.
- Collaborated in Agile environments for requirement analysis, sprint execution, and retrospective reviews, ensuring timely delivery and alignment with financial compliance standards.

Environment: Java 7, J2EE, RESTful, IBM MQ, JSP, HTML, Typescript, XML, Hibernate, Spring Boot, Angular, XSLT, CSS, Oracle 10g, SQL, Microservices, Spring MVC, GCP, Big Query.

Inno Minds: May 2014 - November 2015 (Location: Pune, India)

Java Developer

Responsibilities:

- Transformed the application from a monolithic to a microservices architecture using Spring Boot & Spring Cloud, enabling comprehensive advantages of the cloud ecosystem. This architectural shift significantly improved the scalability and maintainability of the e-learning platform.
- Connected the recommendation engine with Amazon's systems through RESTful APIs, creating seamless channels for data communication and exchange. The integration enhanced the personalized learning experience by leveraging Amazon's robust recommendation algorithms.
- Created PostgreSQL database schemas and developed optimized queries within Java-based microservices on Amazon ECS, supporting efficient data processing and interactions with Amazon RDS PostgreSQL instances. This optimization resulted in faster course content delivery and improved overall system performance.
- Developed Kafka connectors to enable real-time data streaming between Kafka topics and PostgreSQL databases, ensuring timely synchronization and providing immediate business insights. The real-time capabilities facilitated instant updates to user progress and learning analytics.
- Utilized Amazon SQS for task queuing and asynchronous processing, managing resource-intensive tasks without compromising application responsiveness. This implementation improved the system's ability to handle high loads during peak usage periods, such as exam seasons.
- Implemented robust access control by configuring AWS IAM, carefully managing user permissions and securing access to critical AWS resources. The enhanced security measures ensured the protection of sensitive student data and intellectual property.
- Supported and scaled containerized applications using Amazon ECS and EC2 instances, maintaining high system availability and implementing comprehensive fault tolerance strategies. This approach allowed for seamless handling of traffic spikes during course launches and promotional events.
- Designed and implemented a microservices-based content management system using Spring Boot, enabling modular development and easier maintenance of the e-learning platform. The new architecture allowed for rapid updates and additions to course materials without affecting the entire system.
- Integrated OAuth 2.0 and JWT for secure user authentication and authorization across microservices, ensuring protected access to student profiles and course content. This security implementation enhanced user trust and compliance with data protection regulations.
- Developed a custom monitoring solution using Amazon CloudWatch and Prometheus, providing real-time insights into system performance and user engagement metrics. The monitoring system allowed for proactive issue resolution and data-driven improvements to the learning experience.

Environment: Java 1.5, J2EE, JSP, HTML, PostgreSQL, RESTful API, AWS, Hibernate, Kafka, Netflix Eureka, Microservices, Spring Boot, IAM, ECS, EC2, SQS.

Education

Bachelor of Technology – Computer Science *08/2010 – 08/2014*
Malla Reddy Institute of Technology, *Hyderabad, India*
GPA